



PROYECTO FINAL

I PORTADA

UNIVERSIDAD TÉCNICA DE AMBATO

Facultad de Ingeniería en Sistemas, Electrónica e Industrial
“Proyecto Académico de Fin de Semestre: Enero – Julio 2026”

Tema:	Sistema inteligente de radar vehicular: medición de velocidad multi-vehículo, reconocimiento de placas ecuatorianas y sanción difusa proporcional
Carrera:	Ingeniería de Software
Unidad de Organización Curricular:	PROFESIONAL
Línea de Investigación:	Tecnologías de la Información y Sistemas de Control
Nivel y Paralelo:	Séptimo “A”
Alumnos participantes:	Paul Velastegui David Manjarres
Asignatura:	Inteligencia Artificial
Docente:	Ing. Rubén Nogales, Mg.

II INFORME DEL PROYECTO

0.1 Título

Sistema inteligente de radar vehicular para la medición simultánea de velocidad de múltiples vehículos, el reconocimiento de placas ecuatorianas y la evaluación difusa de sanciones proporcionales en un entorno universitario controlado.

0.2 Resumen

El proyecto implementa un radar vehicular inteligente orientado al control de velocidad en una zona universitaria. La solución procesa video en tiempo real o desde archivo y rastrea *simultáneamente* a todos los vehículos del cuadro mediante un detector YOLO11 acoplado al algoritmo de seguimiento ByteTrack, lo que permite medir la velocidad de entre uno y seis vehículos a la vez. La velocidad se estima cuando el vehículo cruza dos líneas virtuales separadas por una distancia conocida. Para cada vehículo detectado se localiza la placa con un modelo YOLO especializado; el texto de la placa se extrae mediante una cadena de procesamiento que combina técnicas clásicas de visión por computador —escala de grises, realce adaptativo CLAHE, umbralización adaptativa gaussiana, aislamiento de banda y análisis de componentes conexos— con un clasificador de caracteres propio basado en la arquitectura



ResNet18 [6] modificada para entrada en escala de grises y 36 clases alfanuméricas. El sistema también integra, como backend primario de OCR, modelos ONNX externos de segmentación y clasificación que amplían la generalización sobre placas de aspecto variado. La lectura se estabiliza por votación temporal ponderada entre fotogramas y se valida con el formato ecuatoriano de tres letras y tres o cuatro dígitos. La decisión disciplinaria se realiza con un sistema difuso de dos etapas: una clasificación Mamdani (felicitación / normal / multa) [13] y una inferencia Takagi–Sugeno [14] que convierte el exceso sobre el límite de 20 km/h en horas de indisponibilidad de forma suave, monótona y proporcional. El sistema registra los eventos, almacena evidencias visuales, notifica por correo electrónico y expone una interfaz web de monitoreo en tiempo real.

0.3 Palabras clave

Visión por computador, seguimiento multi-objeto, reconocimiento de placas vehiculares, ResNet18, CLAHE, componentes conexos, YOLO, ByteTrack, lógica difusa, Mamdani, Takagi–Sugeno, ONNX, radar vehicular.

0.4 Objetivos

General:

- Desarrollar un sistema inteligente de radar vehicular que mida la velocidad de varios vehículos de forma simultánea, reconozca sus placas ecuatorianas y sugiera una sanción proporcional mediante lógica difusa, integrando todo en una aplicación web de monitoreo en tiempo real.

Específicos:

- Implementar el seguimiento multi-vehículo con YOLO11 y ByteTrack para medir la velocidad de cada vehículo al cruzar dos líneas virtuales calibradas.
- Reconocer la cadena alfanumérica de la placa combinando preprocesamiento de imagen clásico (CLAHE, umbralización adaptativa, análisis de componentes conexos) con un clasificador ResNet18 propio de 36 clases y decodificación posicional, con validación del formato vehicular ecuatoriano.
- Integrar modelos ONNX externos de segmentación y clasificación de caracteres como backend primario de OCR, ejecutándolos con `onnxruntime` sin dependencia de TensorFlow, garantizando portabilidad y separación de uso de GPU.
- Diseñar un sistema de inferencia difusa en dos etapas que clasifique la conducta vehicular y calcule una sanción estrictamente proporcional al exceso de velocidad mediante Takagi–Sugeno.
- Presentar los eventos, las evidencias y el razonamiento difuso en una interfaz web con historial y notificación automática por correo de las infracciones.



0.5 Modalidad

Presencial.

0.6 Listado de equipos, materiales y recursos

- Computador con GPU NVIDIA (aceleración CUDA para los detectores YOLO y entrenamiento CNN).
- Cámara web / teléfono como fuente de video en tiempo real; videos de tránsito del campus universitario para pruebas fuera de línea.
- Entorno de desarrollo Python 3.14 con entorno virtual aislado.
- Conjuntos de datos públicos: EMNIST Letters, MNIST y *Dataset_OCR_Placas* de Roboflow Universe (CC BY 4.0) para el clasificador de caracteres; *License Plate Recognition* v13 (Roboflow, CC BY 4.0) para el detector de placa.
- Herramientas de escritura científica: $\text{\LaTeX}$ con pdflatex para el informe.

Dependencias de Python utilizadas

La Tabla 1 detalla cada dependencia, su función en el ecosistema y el uso concreto asignado en el sistema.



Tabla 1: Dependencias del sistema: función y uso concreto.

Dependencia	Qué hace	Cómo se usó en el sistema
ultralytics	Implementación de modelos YOLO y de ByteTrack.	Carga <code>yolo11n</code> (COCO) para detectar y rastrear todos los vehículos del cuadro, y el modelo especializado (<code>best.pt</code>) para localizar la placa.
onnxruntime	Motor de inferencia ONNX independiente del framework de entrenamiento.	Ejecuta en CPU los modelos ONNX externos de segmentación y clasificación de caracteres (backend primario de OCR) sin requerir TensorFlow; evita conflictos de memoria GPU con PyTorch.
torch / torchvision	Backend de cómputo GPU y colección de modelos y datasets de visión.	Implementa la arquitectura ResNet18 del clasificador propio, carga los datasets EMNIST y MNIST para entrenamiento, y aceleración CUDA para YOLO.
opencv-python	Procesamiento de imagen y video clásico.	Captura de cámara y archivo; conversión a escala de grises, CLAHE, umbralización adaptativa gaussiana, morfología, componentes conexos, aislamiento de banda, corrección de nitidez, upscaling bicúbico y dibujado del HUD.
numpy	Cálculo numérico sobre arreglos multidimensionales.	Representa imágenes y máscaras; construye los <i>batches</i> de entrada a la CNN y a ONNX; realiza la votación temporal y los cálculos del sistema difuso.
scikit-fuzzy	Funciones de pertenencia difusa e interpolación.	Define las curvas triangulares y trapezoidales de velocidad para la clasificación Mamdani y los sub-niveles de sanción Takagi-Sugeno.
fastapi / uvicorn	Framework y servidor web ASGI.	API REST (iniciar/detener, configurar fuente y distancia), streaming MJPEG al navegador y emisión de eventos al frontend vía WebSocket.
matplotlib	Generación de figuras en memoria.	Renderiza el razonamiento difuso (curvas de pertenencia y curva de sanción) para adjuntar en el correo y mostrar en el modal del evento.

TAC (Tecnologías para el Aprendizaje y Conocimiento):

- ☐ Plataformas educativas
- ☒ Aplicaciones educativas
- ☒ Inteligencia Artificial
- ☐ Recursos audiovisuales



0.7 Introducción

El control de velocidad vehicular mediante visión por computador representa un área activa de investigación en sistemas inteligentes de transporte. La identificación automática de placas vehiculares (ALPR) se ha consolidado como tecnología base para vincular una infracción con el vehículo responsable [1]. En entornos universitarios, donde la velocidad permitida es baja y el tráfico peatonal es intenso, un sistema de este tipo puede generar evidencia objetiva ante conductas de riesgo sin requerir intervención humana permanente. Trabajos previos han integrado visión artificial para el control de ingreso vehicular en parqueaderos universitarios [2] y para la detección de placas a partir de imágenes estáticas [3].

Desde el punto de vista técnico, los sistemas ALPR se organizan en etapas: adquisición, localización de la placa, segmentación de caracteres y reconocimiento final [1]. Los detectores de objetos de una sola etapa, en particular la familia YOLO, han demostrado velocidad y precisión adecuadas para el procesamiento en tiempo real [4]. El seguimiento de múltiples objetos con ByteTrack permite mantener identidades estables bajo variaciones de confianza de detección [5]. El reconocimiento de caracteres puede abordarse mediante técnicas clásicas de segmentación (umbralización adaptativa, componentes conexos) combinadas con clasificadores basados en redes residuales [6], o mediante modelos de segmentación aprendida como la U-Net [7]. La decisión de sanción, por su parte, se beneficia de la lógica difusa cuando las variables son continuas y graduales [12, 13, 14].

El problema abordado es construir un radar que, a partir de una sola cámara, sea capaz de: (i) medir la velocidad de *múltiples* vehículos simultáneamente; (ii) leer de forma confiable la placa ecuatoriana de cada vehículo pese a la distancia, el desenfoque por movimiento y la variación de ángulo; y (iii) traducir el exceso de velocidad en una sanción proporcional e interpretable.

0.8 Marco de referencia: formato de placas ecuatorianas

El sistema se diseña para placas ecuatorianas de seis o siete caracteres: tres letras mayúsculas seguidas de tres o cuatro dígitos (p. ej. ABC-1234). Ecuador dispone de dos modelos en circulación: el modelo actual de fondo blanco con letras azules e identificador «ECUADOR» en la franja superior, y el modelo anterior de fondo blanco liso. Ambos comparten la misma estructura alfanumérica [3]. Además de los caracteres principales, las placas contienen elementos visuales que no deben reconocerse como caracteres: el encabezado «ECUADOR», el logotipo de la Agencia Nacional de Tránsito (ANT), los orificios de tornillo y el separador. El sistema aprovecha la estructura de tres letras + dígitos como capa de validación al final del pipeline, descartando cualquier lectura que no satisfaga esa expresión regular.

0.9 Metodología

0.9.1 Arquitectura general del sistema

El sistema se organiza en cuatro bloques funcionales: adquisición y seguimiento de video, pipeline de visión para el reconocimiento de placas, módulo de decisión difusa y backend con frontend web de monitoreo. La Figura 1 muestra el flujo completo desde la captura del video hasta el registro del evento.

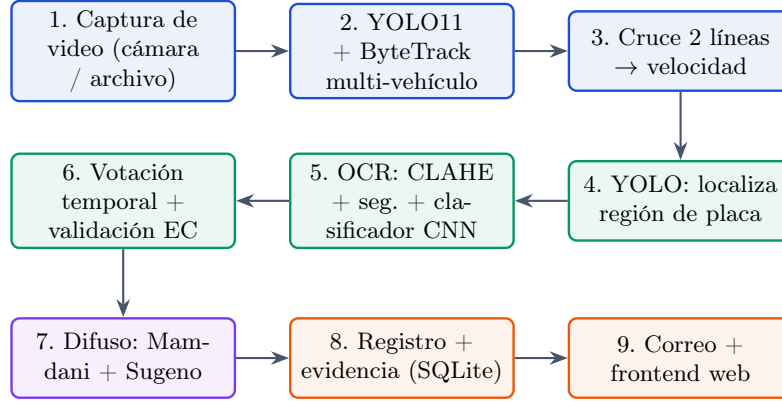


Figura 1: Flujo general del radar vehicular: del video al evento registrado.

Fuente: Elaboración propia.

0.9.2 Dominios de muestreo de la cámara

Dominio espacial: resolución y detección. La detección de la placa opera sobre el recorte del vehículo extraído del fotograma a su resolución completa. El recorte de placa resultante se envía al módulo de OCR sin reducción de resolución. Cuando su altura es inferior a 72 píxeles (vehículo lejano), se aplica un escalado bicúbico antes del reconocimiento: la interpolación bicúbica preserva mejor los bordes de los caracteres que la bilineal o la de vecino más próximo. El detector YOLO procesa internamente la imagen a un tamaño fijo (416×416), pero la extracción del recorte siempre se realiza sobre el fotograma original de alta resolución.

Dominio temporal: velocidad. La zona de medición se define con dos líneas virtuales con una distancia real calibrada d . El punto de referencia del vehículo es el centro inferior de su caja delimitadora. Al cruzar la primera línea se registra el tiempo t_a ; al cruzar la segunda, t_b . La velocidad es:

$$v = \frac{d}{t_b - t_a} \times 3,6 \quad [\text{km/h}]. \quad (1)$$

Para fuentes de archivo se emplean las marcas de tiempo del video, no el reloj de CPU, lo que garantiza exactitud aunque el procesamiento sea más lento que el tiempo real. El número de fotogramas observados durante el cruce, $N = (t_b - t_a) \times F_s$, determina la confiabilidad de la medición: con pocos fotogramas el error relativo es mayor, en concordancia con el criterio de Nyquist en el dominio temporal [9].



0.9.3 Seguimiento multi-vehículo (YOLO11 + ByteTrack)

Cada fotograma se escala al 50 % y se procesa con YOLO11 nano preentrenado en COCO [4] (clases relevantes: automóvil, motocicleta, bus, camión). Las detecciones se asocian entre fotogramas con ByteTrack [5], un algoritmo de asociación que mantiene identidades estables incluso cuando la confianza de detección es baja, diferenciando vehículos temporalmente ocultos (*lost*) de los que siguen en tránsito. ByteTrack utiliza la intersección sobre unión (IoU) entre cajas detectadas y trayectorias predichas para la asociación, y la distancia de Kalman para predecir la posición del vehículo cuando no se detecta en un fotograma concreto.

Gracias a las identidades persistentes, el sistema mantiene un estado independiente por vehículo: cruces de líneas, tiempos, lecturas de placa acumuladas. Esto habilita la medición simultánea de varios vehículos sin cruzar sus datos. Una vez completado el cruce se inicia el pipeline de lectura de placa; al finalizar, el evento se registra con la placa leída (o con la etiqueta SIN-LECTURA si el OCR no converge), la velocidad y la sanción difusa, garantizando que ningún vehículo medido quede sin registro.

0.9.4 Detección de la placa vehicular (YOLO)

La región de la placa se localiza con un modelo YOLO de una sola clase (*License_Plate*), operando sobre el recorte del vehículo para reducir el área de búsqueda y minimizar falsos positivos por elementos del fondo. La caja resultante se amplía un 8 % en cada borde para incluir los caracteres que puedan estar en el límite de la detección, antes de enviarla al módulo de OCR.

0.9.5 Pipeline de reconocimiento de caracteres (OCR)

El módulo de OCR recibe el recorte de placa y produce el texto crudo de la cadena de caracteres. El sistema emplea dos backends OCR con selección automática:

1. **Backend primario (ONNX):** modelos externos de mayor generalización, ejecutados con `onnxruntime` (ver sección 0.9.6).
2. **Backend secundario (CNN propia):** clasificador ResNet18 con segmentación clásica por visión por computador (ver sección 0.9.7), que actúa como respaldo cuando el backend primario no produce texto válido.

La selección se controla con la variable de entorno `OCR_BACKEND` (valor predeterminado: `amigo` para el backend ONNX; `cnn` para forzar el clasificador propio). La conmutación automática entre backends garantiza siempre un resultado: primero se intenta el más generalizable; si no converge, se recurre al propio, que tiene mayor exhaustividad (*recall*).



0.9.6 Backend ONNX: modelos externos integrados

Como backend primario de OCR se integran dos modelos entrenados externamente sobre un corpus internacional de mayor diversidad: un modelo de segmentación semántica de caracteres de arquitectura U-Net [7] y un clasificador CNN de 36 clases. Ambos se ejecutan mediante `onnxruntime` [8] en CPU:

- **Segmentador U-Net** (`amigo_seg_unet.onnx`): recibe la imagen de placa en escala de grises a $96 \times 256 \times 1$ px y produce una máscara de probabilidad por píxel. La máscara se umbraliza en 0.5; a partir de las regiones activas se extraen los contornos y se generan las cajas de cada carácter. La segmentación aprendida (frente a la umbralización fija) generaliza mejor ante iluminación variable, fondos complejos y variaciones de tipografía.
- **Clasificador CNN externo** (`amigo_ocr_cnn.onnx`): recibe cada recorte de carácter a $64 \times 64 \times 1$ px y produce un vector de 36 probabilidades softmax. El modelo fue entrenado sobre un corpus que incluye placas de múltiples países, lo que le confiere mayor cobertura tipográfica que el clasificador propio entrenado únicamente con EMNIST, MNIST y un dataset nacional.

La ventaja principal de esta integración es que TensorFlow no es requerido en el entorno de ejecución (Python 3.14 no dispone de ruedas TensorFlow), evitando conflictos de versiones y liberando la GPU exclusivamente para el seguimiento YOLO.

0.9.7 Backend propio: segmentación clásica + ResNet18

El backend propio realiza el reconocimiento de caracteres mediante una cadena de procesamiento clásico seguida de un clasificador neuronal entrenado por el equipo.

Etapas 1: preprocesamiento. El recorte de placa en color pasa por:

1. **Escala de grises.** Elimina la información de color irrelevante.
2. **Nitidez adaptativa.** Se mide la varianza del Laplaciano; si esta es inferior a 80 (desenfoco por movimiento presente), se aplica un filtro de nitidez de 3×3 que refuerza las altas frecuencias y recupera parcialmente los bordes de los caracteres.
3. **CLAHE** (*Contrast Limited Adaptive Histogram Equalization*). El histograma se ecualiza localmente en bloques de 4×4 px con un límite de *clip* de 3.0, compensando las variaciones de iluminación que una ecualización global no podría corregir sin amplificar el ruido en zonas uniformes.

Etapas 2: aislamiento de la banda de caracteres. Las placas actuales contienen el encabezado «ECUADOR», el logo ANT y tornillos que, sin filtrar, generan falsos positivos en la segmentación. El sistema los elimina mediante un análisis de componentes conexos de dos fases:



- **Fase CC (componentes conexos):** la imagen se normaliza a 64 px de alto, se binariza con umbral adaptativo gaussiano local (tamaño de bloque entre 11 y 51 px según la altura, sustracción de constante 9) y se analizan los componentes. Los candidatos a carácter deben tener: altura entre el 30 % y el 97 % de la imagen, relación ancho/alto entre 0.08 y 1.4, y área superior al 0.6 % del área total. Con los candidatos válidos se calcula la mediana de sus posiciones verticales y se recorta la franja correspondiente.
- **Fase de proyección (respaldo):** cuando la fase CC no produce candidatos suficientes (placas con fondo muy oscuro o caracteres fusionados), se calcula la densidad de píxeles blancos por fila (proyección horizontal), se identifican los bloques activos y se selecciona el de mayor puntuación (densidad $\times \sqrt{\text{longitud}}$).

Etapla 3: segmentación de caracteres individuales. Sobre la banda aislada se aplica umbral adaptativo gaussiano y apertura morfológica (2×2) para eliminar artefactos de ruido. Los componentes conexos resultantes se filtran y se agrupan por mediana de alturas y centros verticales para descartar residuos del logo o tornillos. Los componentes supervivientes se ordenan de izquierda a derecha. Si algún componente es anómalamente ancho ($> 1,5 \times$ el ancho mediano), se divide en partes iguales para recuperar los caracteres fusionados.

Etapla 4: clasificador ResNet18 propio. Cada recorte de carácter se clasifica con la red residual desarrollada por el equipo. La arquitectura base es ResNet18 [6], adaptada mediante las modificaciones de la Tabla 2.

Tabla 2: Adaptaciones de ResNet18 para el clasificador propio de caracteres.

Componente	Modificación respecto a ResNet18 estándar
Primera conv.	Conv2D(1→64, 3×3 , stride 1, padding 1, sin bias): un canal de entrada (escala de grises) en lugar de tres (RGB); kernel 3×3 en lugar del 7×7 original para preservar detalle en imágenes de 48×48 px.
MaxPool inicial	Reemplazado por Identity : se suprime el maxpooling que reduciría a la mitad el mapa de características, lo que es excesivo para caracteres de baja resolución donde cada píxel es relevante.
Bloques residuales	Sin modificaciones: cuatro grupos (64, 128, 256, 512 canales) de dos bloques cada uno; cada bloque con dos capas Conv2D + BN + ReLU y una conexión de atajo para propagar el gradiente.
Capa de salida	Dropout(0.3) + Linear(512→36) : dropout para regularización antes de la capa densa final de 36 logits.

La red recibe recortes redimensionados a 48×48 px (entrenamiento real) o 32×32 px (entrenamiento base), normalizados entre 0 y 1. Todos los recortes de un fotograma se procesan en un único *batch*



GPU.

Etapas 5: decodificación posicional. Las probabilidades softmax por carácter se decodifican restringiendo la búsqueda al subconjunto correcto según la posición: letras (A–Z) en las posiciones 1–3, dígitos (0–9) en las posiciones 4–7. Esta restricción posicional elimina sin ambigüedad las confusiones entre caracteres de grupos distintos: $O \leftrightarrow 0$, $I \leftrightarrow 1$, $Z \leftrightarrow 2$, $B \leftrightarrow 8$, $S \leftrightarrow 5$. Si el número de recortes supera lo esperado por la segmentación de elementos espurios, se desliza una ventana de 7 y luego de 6 posiciones, eligiendo la de mayor confianza media.

0.9.8 Validación de formato y votación temporal

El vehículo ocupa la zona de medición durante múltiples fotogramas. En cada fotograma procesado el OCR produce una lectura cruda. El sistema acumula estas lecturas por vehículo (identificado por su ID de ByteTrack) y produce un consenso por votación posición a posición: gana el carácter con más votos en cada posición; los empates se resuelven por la confianza del clasificador. Los fotogramas donde la placa ocupa mayor área (vehículo más cercano) reciben mayor peso en la votación. Sobre el consenso se aplica la validación de formato ecuatoriano (expresión regular $[A-Z]\{3\}[0-9]\{3,4\}$); sólo se registra un evento cuando el consenso satisface esta condición.

0.9.9 Sistema de inferencia difusa

La decisión disciplinaria se modela con lógica difusa [12], adecuada cuando la velocidad es una variable continua y la sanción debe reflejar esa continuidad en lugar de escalones rígidos. El diseño se realiza en dos etapas:

1. **Clasificación (Mamdani) [13].** Tres conjuntos difusos de velocidad determinan la categoría del evento por máxima pertenencia: *felicitación*, *normal* y *multa*. La pertenencia *multa* es una función trapezoidal monótona creciente desde el límite de 20 km/h, produciendo una rampa limpia sin el perfil en «serrucho» que surgía del máximo de varios triángulos solapados.
2. **Sanción (Takagi–Sugeno) [14].** Sólo cuando la categoría es *multa*, las horas de indisponibilidad se calculan mediante cinco sub-niveles de severidad que cubren el rango 20–40 km/h con solapamiento del 50 %. Cada sub-nivel tiene un consecuente *singleton* creciente (c_i). La sanción final es el promedio ponderado:

$$H(v) = \frac{\sum_{i=1}^5 \mu_i(v) c_i}{\sum_{i=1}^5 \mu_i(v)}, \quad (2)$$

donde $\mu_i(v)$ es el grado de pertenencia de la velocidad v al sub-nivel i . La combinación convexa de singletons estrictamente crecientes garantiza que $H(v)$ sea monótona y suave: cualquier incremento de v produce un incremento de H .

La Tabla 3 resume las funciones de pertenencia de ambas etapas con sus parámetros exactos tal como se implementaron.

Tabla 3: Variables difusas del sistema: categorías Mamdani y sub-niveles Sugeno.

Etapa	Conjunto	Función / Parámetros	Consecuente
Mamdani	Felicitación	Trapezoidal: [0, 0, 8, 11] km/h	Sin sanción
	Normal	Triangular: [9, 15, 21] km/h	Sin sanción
	Multa	Trapezoidal: [20, 25, 40, 40] km/h	→ Sugeno
Sugeno	Leve	Triangular: [20, 24, 28] km/h	$c_1 = 5$ h
	Moderada	Triangular: [24, 28, 32] km/h	$c_2 = 18$ h
	Alta	Triangular: [28, 32, 36] km/h	$c_3 = 34$ h
	Grave	Triangular: [32, 36, 40] km/h	$c_4 = 52$ h
	Extrema	Trapezoidal: [36, 40, 40, 40] km/h	$c_5 = 70$ h

Justificación de Takagi–Sugeno frente a Mamdani para la sanción. Cuando la sanción se modela como un conjunto difuso de salida con el defuzzificador de centroide de Mamdani, la curva $H(v)$ presenta mesetas y micro-descensos no monótonos en los bordes entre conjuntos (efecto «serrucho»): en ciertos rangos, mayor velocidad produce menor sanción, violando el principio de proporcionalidad requerido por las regulaciones de tránsito [15]. El promedio ponderado de singletons (2) elimina completamente este efecto: los singletons crecientes garantizan la monotonicidad matemáticamente, sin necesidad de defuzzificación por integración numérica.

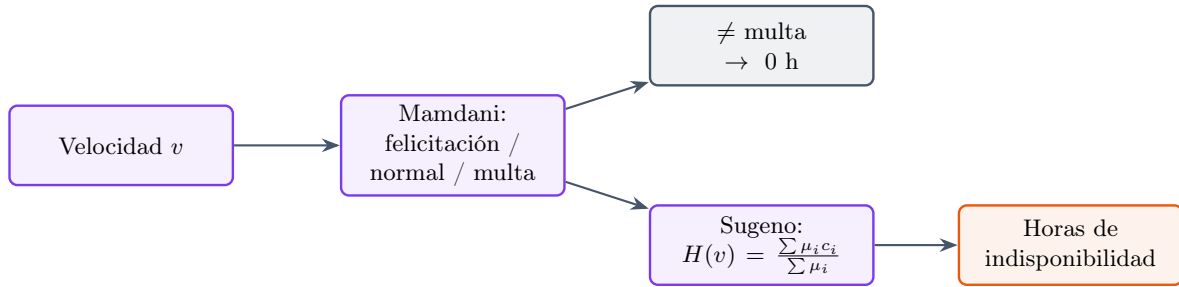


Figura 2: Decisión difusa en dos etapas: clasificación Mamdani y sanción proporcional Takagi–Sugeno.

Fuente: Elaboración propia.

0.9.10 Backend, frontend y notificación

El backend está construido con FastAPI y Uvicorn. Integra el pipeline de visión, transmite el video procesado al navegador en formato MJPEG y emite eventos al frontend vía WebSocket. Por cada evento registrado el sistema: (i) guarda el fotograma capturado y las evidencias visuales en disco; (ii) registra el evento en una base de datos SQLite con la placa, la velocidad, la clasificación difusa, las horas de sanción y la marca de tiempo; y (iii) dispara una notificación por correo con las evidencias y el gráfico del razonamiento difuso (generado con matplotlib en memoria). La distancia entre líneas y la fuente de video son configurables desde el frontend sin editar código. El frontend permite seleccionar la fuente



(cámara, archivo o URL RTSP), arrastrar las líneas de medición con el ratón, observar el video en vivo con *bounding boxes* y etiquetas de velocidad, y revisar el historial de eventos con miniaturas y detalle por evento.

0.10 Conjuntos de datos y entrenamiento

0.10.1 Detector de placa (YOLO)

El detector de placa se entrenó con el conjunto público *License Plate Recognition* v13 de Roboflow Universe (CC BY 4.0) [10], de origen internacional con una sola clase (*License_Plate*). El conjunto contiene 101 866 imágenes generadas por aumento a partir de unas 7 276 imágenes fuente (volteo, recorte, rotación, cizalla, brillo, exposición y desenfoque). La Tabla 4 muestra el reparto por partición.

Tabla 4: Conjunto de datos para el detector de placas [10].

Partición	Imágenes
Entrenamiento	98 798
Validación	2 048
Prueba	1 020
Total	101 866

0.10.2 Clasificador ResNet18 propio: datos y estrategia de entrenamiento

El clasificador se entrenó en dos fases sucesivas sobre datos de distinto origen y nivel de dominio:

Fase base: EMNIST Letters + MNIST. La primera fase utiliza los datasets estándar de dígitos y letras manuscritas: EMNIST Letters (A–Z, aproximadamente 145 600 muestras de entrenamiento, 24 800 de prueba) y MNIST (0–9, 60 000 de entrenamiento, 10 000 de prueba). Las clases de EMNIST se reindexan a 0–25 y las de MNIST a 26–35 para formar un espacio unificado de 36 clases. El modelo se entrena con entrada a 32×32 px, 25 épocas, función de pérdida de entropía cruzada con *label smoothing* de 0.1 (reduce la sobreconfianza del modelo en ejemplos fáciles), optimizador AdamW ($lr=10^{-3}$, $wd=10^{-4}$) y planificador OneCycleLR con calentamiento del 10 % de las épocas y decrecimiento coseno. La augmentación incluye: rotación ($\pm 15^\circ$), traslación (12 % por eje), cizalla (10°), escala (0,82–1,18 $\times$), distorsión de perspectiva (prob. 0.5), desenfoque gaussiano (prob. 0.4) y **RandomErasing** (prob. 0.25) para simular oclusiones y suciedad.

Fase real: Dataset_OCR_Placas. La segunda fase reentrena el modelo sobre glifos reales extraídos de placas vehiculares (*Dataset_OCR_Placas*, Roboflow Universe, CC BY 4.0) [11]. La entrada se amplía a 48×48 px (mayor preservación del detalle de los trazos), las épocas aumentan a 70 y se activa la precisión mixta automática (AMP) para aprovechar la GPU eficientemente. La augmentación se ajusta al dominio real: rotación ($\pm 8^\circ$), traslación (8 % por eje), cizalla (5°), escala (0,90–1,10 $\times$),



distorsión de perspectiva (prob. 0.4), variación de brillo y contraste (30 %), desenfoque gaussiano y **RandomErasing** (prob. 0.2) para simular tornillos, suciedad o bordes recortados. La función de pérdida mantuvo entropía cruzada con *label smoothing* de 0.08 y el optimizador OneCycleLR con calentamiento del 15 %.

La Tabla 5 resume el volumen de la fase base:

Tabla 5: Volumen del dataset base (EMNIST + MNIST) para el clasificador ResNet18.

Dataset	Train	Test
EMNIST Letters (A–Z)	145 600	24 800
MNIST (0–9)	60 000	10 000
Total	205 600	34 800

0.11 Resultados y discusión

0.11.1 Detector de placa

El detector YOLO alcanzó los valores de la Tabla 6. La precisión de 0.990 y el mAP@50 de 0.950 confirman que la región de la placa se localiza de forma confiable sobre el recorte del vehículo. El mAP@50–95 de 0.661 refleja la exigencia del ajuste fino de la caja, pero es suficiente para el propósito: el OCR opera sobre el recorte extraído a resolución completa del fotograma original, no sobre la caja reducida del detector.

Tabla 6: Métricas del detector YOLO de placas.

Métrica	Valor
Precisión	0.990
Exhaustividad (<i>recall</i>)	0.924
mAP@50	0.950
mAP@50–95	0.661

0.11.2 Clasificador ResNet18 propio

El clasificador ResNet18 adaptado se evaluó sobre el conjunto de prueba de EMNIST+MNIST (34 800 muestras) y sobre recortes de placas del campus universitario. La Tabla 7 muestra las métricas globales en el conjunto estándar.

Tabla 7: Métricas globales del clasificador ResNet18 de 36 clases.

Métrica	Precisión	Recall	F1
Promedio macro	0.914	0.904	0.907
Promedio ponderado	0.914	0.913	0.912
Exactitud global	0.913		

Las clases con menor F1 individual corresponden a caracteres visualmente ambiguos (O/0, I/1), efecto esperado y mitigado en la práctica por la decodificación posicional. La comparación exploratoria de resoluciones (Tabla 8) justificó la elección de 48×48 px para la fase de reentrenamiento real.

Tabla 8: Exactitud del clasificador ResNet18 según resolución de entrada.

Resolución	Exactitud
32×32	0.813
48×48	0.913

0.11.3 Funcionamiento del radar en eventos reales

La Figura 3 muestra el dashboard de monitoreo en tiempo real con las líneas de medición, el panel de configuración de la fuente, las etiquetas de velocidad y el historial de eventos.

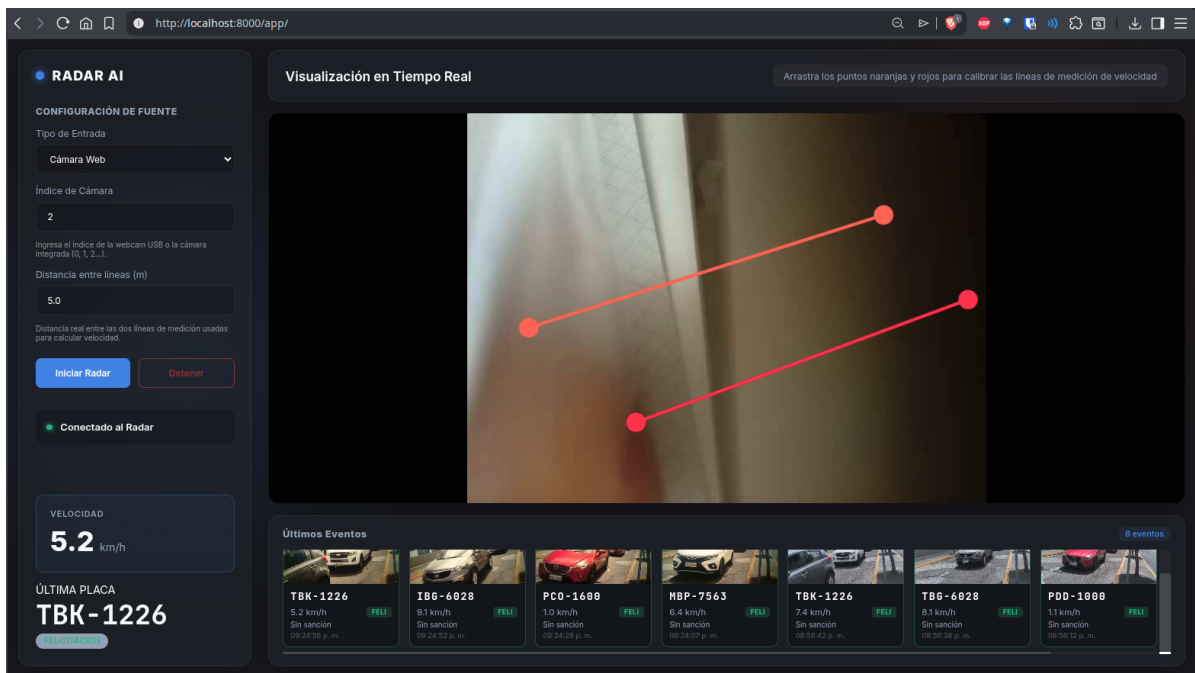


Figura 3: Dashboard de monitoreo en tiempo real: líneas de medición, panel de fuente y historial de eventos.

Fuente: Elaboración propia.

En una evaluación sobre video de tránsito del campus, el radar midió la velocidad de cuatro vehículos simultáneamente y registró los cuatro con su placa leída de forma reproducible en ejecuciones sucesivas. La Figura 4 muestra el detalle de un evento individual: placa, velocidad, clasificación difusa y razonamiento.

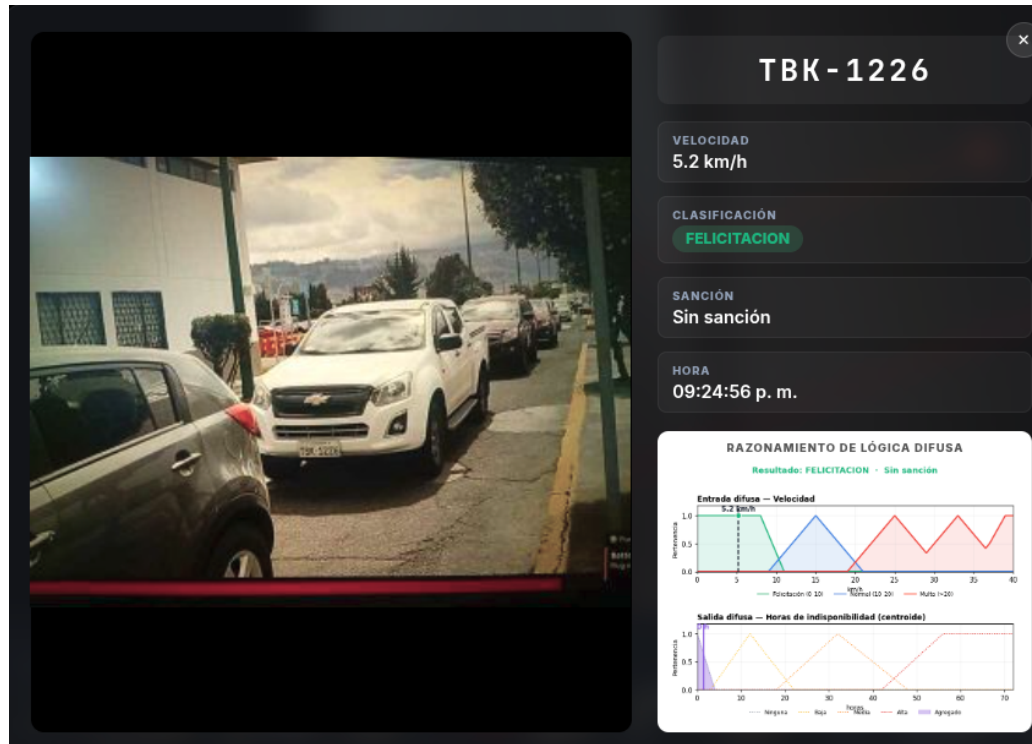


Figura 4: Detalle de un evento: placa, velocidad medida, clasificación difusa y razonamiento.
Fuente: Elaboración propia.

0.11.4 Sanción difusa proporcional

La Figura 5 muestra el razonamiento difuso para un evento de multa. El panel superior presenta las tres curvas de pertenencia de velocidad (felicitación, normal, multa); el panel inferior muestra la curva $H(v)$, nula en la zona permitida (≤ 20 km/h) y creciente de forma suave y monótona hasta 70 horas a 40 km/h. La Tabla 9 confirma la proporcionalidad estricta con valores representativos.

Resultado: MULTA · 1 día con 2 horas

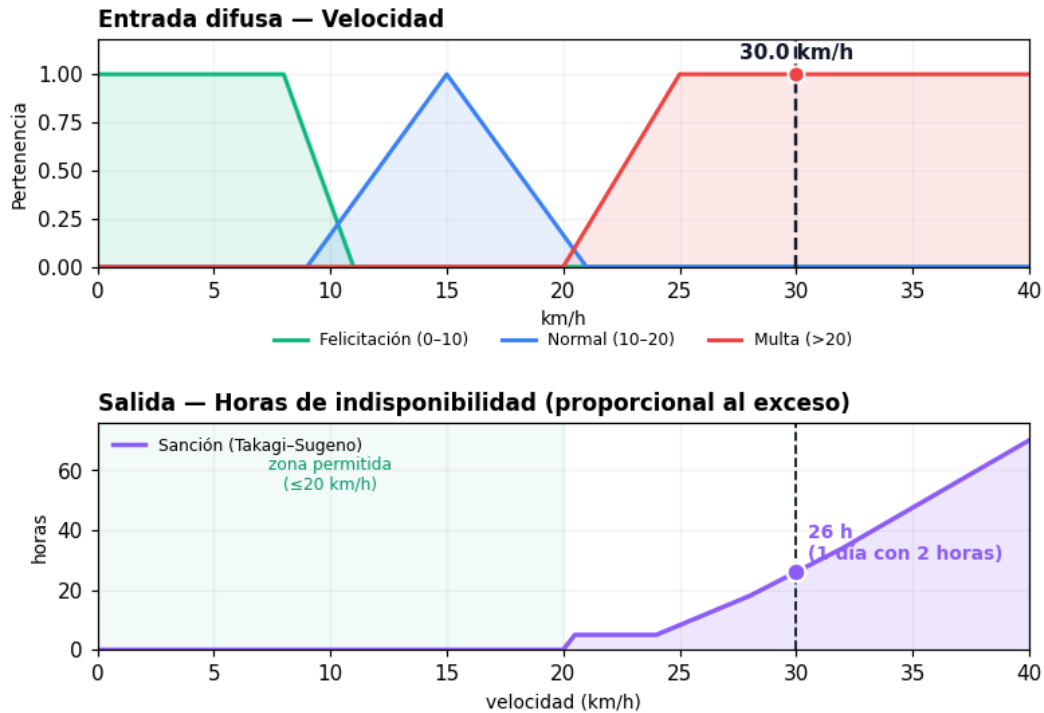


Figura 5: Razonamiento difuso: curvas de pertenencia de velocidad (arriba) y curva de sanción proporcional $H(v)$ (abajo).

Fuente: Elaboración propia.

Tabla 9: Sanción proporcional según velocidad (Takagi–Sugeno).

Velocidad	Horas	Equivalente
≤ 20 km/h	0	Sin sanción
21 km/h	5	5 horas
25 km/h	8	8 horas
30 km/h	26	≈ 1 día con 2 horas
35 km/h	48	2 días
40 km/h	70	≈ 2 días con 22 horas

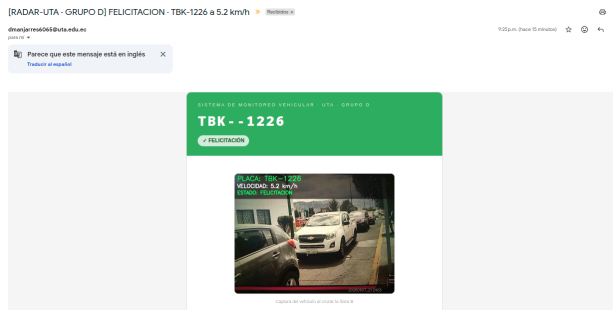
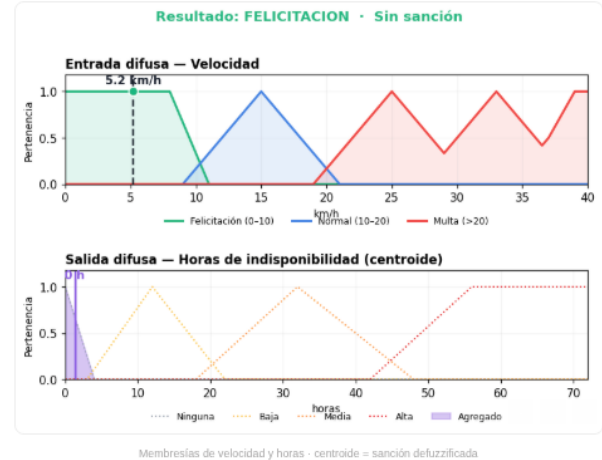
0.11.5 Notificación por correo

Cada infracción genera un correo automático con la evidencia del evento, la placa leída, la velocidad medida, la clasificación difusa y el gráfico de razonamiento. Las Figuras 6 y 6 muestran ejemplos de correos con distintos niveles de riesgo.

DETALLES DEL EVENTO

Velocidad Registrada	5.2 km/h
Clasificación Difusa	✓ FELICITACIÓN
Rango de Velocidad	0 – 10 km/h (Felicitación · zona segura)
Sanción	Sin sanción

RAZONAMIENTO DE LÓGICA DIFUSA



(a)

RADAR AI · UTA Grupo D — Sistema Inteligente de Monitoreo Vehicular
 Manjarres Quintero, David Oswaldo · Velastegui Eugenio, Anthony Paul

(b)

Figura 6: Correos de notificación: (a) evento con riesgo medio, (b) evento con nivel de riesgo distinto.

Fuente: Elaboración propia.

0.11.6 Discusión

El seguimiento con ByteTrack permite el caso multi-vehículo sin confundir identidades, y el estado independiente por vehículo garantiza que ningún evento medido quede sin registro. El módulo de segmentación clásica (CLAHE, umbral adaptativo gaussiano, aislamiento de banda por componentes conexos con proyección horizontal como respaldo, y división de componentes fusionados) demostró adaptarse a variaciones de iluminación y tipo de placa sin parámetros fijos de umbral global.

La decodificación posicional —letras en posiciones 1–3, dígitos en 4–7— resuelve sin ambigüedad las confusiones más frecuentes entre caracteres de grupos distintos, lo que contribuye decisivamente a la exactitud final. La votación temporal absorbe los errores ocasionales de fotogramas individuales con desenfoque o ángulo adverso, elevando la precisión de la lectura final sobre la de cualquier fotograma aislado.

La estrategia de dos backends OCR —el primario ONNX para mayor generalización y el propio ResNet18 como respaldo de mayor recall— demostró ser correcta: el sistema lee placas variadas sin requerir TensorFlow en el entorno de ejecución (Python 3.14), con la GPU dedicada exclusivamente al seguimiento YOLO. La decisión difusa en dos etapas corrige el comportamiento no monótono del centroide



de Mamdani y produce una sanción proporcional, continua y jurídicamente defensible.

0.12 Alcance, generalización y limitaciones

El sistema exhibe una capacidad significativa de generalización más allá de los datos y escenas empleados durante el entrenamiento:

- **Detección y seguimiento.** YOLO11n está preentrenado en COCO [4]; no fue reentrenado con los videos del campus, por lo que detecta vehículos en cualquier escena o cámara sin restricción al dominio del proyecto.
- **Detección de placa.** El detector fue entrenado sobre 101 866 imágenes de origen internacional [10]; localiza la región de la placa de forma consistente ante variaciones de escala, iluminación y ángulo de captura.
- **Reconocimiento de caracteres.** El clasificador ResNet18 propio fue entrenado con EMNIST Letters y MNIST (alfabeto y dígitos universales) y reentrenado con glifos reales de placas vehiculares. La segmentación clásica opera sobre propiedades geométricas invariantes, independiente del tipo de placa. Los modelos ONNX externos, entrenados sobre un corpus internacional de mayor diversidad, actúan como backend primario y amplían la cobertura tipográfica. El hecho de que el sistema lea correctamente placas del campus ausentes del entrenamiento constituye evidencia directa de generalización.

Las limitaciones son:

- **Formato.** La validación impone el formato ecuatoriano (tres letras + tres o cuatro dígitos). Placas de otros países se rechazan deliberadamente.
- **Condiciones ópticas.** La lectura confiable requiere resolución suficiente (vehículo cercano), ángulo aproximadamente frontal y ausencia de desenfoque extremo o suciedad severa.
- **Calibración.** La exactitud de la velocidad depende de la correcta medición de la distancia real entre líneas y de una frecuencia de fotogramas suficiente para capturar el cruce completo.
- **Confusiones residuales.** Persisten errores entre caracteres del mismo grupo morfológicamente similares (C/G, P/R), que la votación temporal mitiga pero no elimina completamente.



0.13 Habilidades blandas empleadas

- Trabajo en equipo
- Responsabilidad
- Pensamiento crítico
- Resolución de problemas

0.14 Conclusiones

- Se desarrolló un radar vehicular funcional que mide la velocidad de varios vehículos simultáneamente mediante YOLO11 y ByteTrack, garantizando que cada vehículo medido quede registrado con su placa y su sanción sugerida.
- La cadena de preprocesamiento clásico (escala de grises, nitidez adaptativa por Laplaciano y CLAHE local) normaliza las condiciones de la placa antes de la segmentación, y el aislamiento de banda por componentes conexos con proyección horizontal como respaldo elimina eficazmente el encabezado «ECUADOR», el logo ANT y los tornillos.
- El clasificador ResNet18 adaptado para escala de grises (48×48 px, sin maxpool inicial, Dropout+Linear→36 clases) alcanzó una exactitud global de 0.913 con métricas macro y ponderadas superiores a 0.90. La decodificación posicional elimina sin ambigüedad las confusiones cruzadas entre grupos de caracteres.
- La arquitectura de dos backends OCR —ONNX externo como primario para maximizar la generalización, y ResNet18 propio como respaldo para maximizar el recall— permite combinar lo mejor de ambos enfoques sin requerir TensorFlow en el entorno de ejecución, con la GPU exclusivamente disponible para el seguimiento.
- El detector YOLO de placas alcanzó una precisión de 0.990 y un mAP@50 de 0.950, localizando de forma confiable la región de la placa sobre el recorte del vehículo en condiciones variadas.
- La votación temporal posición a posición, ponderada por el área de la placa en el fotograma, eleva la precisión de la lectura final muy por encima de la de cualquier fotograma individual.
- El sistema difuso de dos etapas (clasificación Mamdani + sanción Takagi–Sugeno) produce una sanción estrictamente monótona y proporcional al exceso sobre 20 km/h, corrigiendo el comportamiento no monótono del centroide de Mamdani.



0.15 Recomendaciones

- Calibrar la distancia entre líneas y la posición de las líneas virtuales según la geometría real de la escena; una distancia mal medida o una frecuencia de fotogramas insuficiente degradan la exactitud de la velocidad de forma directamente proporcional.
- Ubicar la zona de medición donde el vehículo se observe de frente y a distancia cercana; las placas lejanas generan recortes pequeños que, aun con upscaling bicúbico, tienen menor información disponible para el OCR.
- Ampliar el conjunto de reentrenamiento del clasificador ResNet18 con recortes adicionales de placas ecuatorianas etiquetadas para reducir las confusiones residuales entre caracteres morfológicamente similares del mismo grupo (C/G, P/R, I/J).
- Explorar como trabajo futuro un detector de texto de extremo a extremo (YOLO sobre el recorte de placa) que elimine la etapa de segmentación clásica y reduzca la latencia total del OCR, especialmente para placas con fondo irregular.
- Incorporar la reincidencia como variable de agravamiento en el sistema difuso, consultando el historial de eventos del vehículo antes de emitir la sanción definitiva, lo que aumentaría la proporcionalidad longitudinal del sistema.

0.16 Referencias bibliográficas

- [1] J. Shashirangana, H. Padmasiri, D. Meedeniya, and C. Perera, “Automated license plate recognition: A survey on methods and techniques,” *IEEE Access*, vol. 9, pp. 11 203–11 225, 2021.
- [2] M. A. Álvarez Durán, “Análisis, diseño e implementación de un sistema de control de ingreso de vehículos basado en visión artificial y reconocimiento de placas en el parqueadero de la Universidad Politécnica Salesiana – Sede Cuenca,” Tesis de Ingeniería de Sistemas, Universidad Politécnica Salesiana, Cuenca, Ecuador, 2014.
- [3] Y. J. León Aucapiña, “Diseño e implementación de una aplicación para la detección de placas vehiculares a partir de imágenes,” Trabajo de titulación, Universidad Técnica Particular de Loja, Loja, Ecuador, 2017.
- [4] G. Jocher and J. Qiu, “Ultralytics YOLO11,” Ultralytics, 2024. [En línea]. Disponible: <https://docs.ultralytics.com/models/yolo11/>
- [5] Y. Zhang *et al.*, “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 1–21.
- [6] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.



- [7] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional networks for biomedical image segmentation,” in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 2015, pp. 234–241.
- [8] ONNX Runtime developers, “ONNX Runtime,” Microsoft, 2024. [En línea]. Disponible: <https://onnxruntime.ai/>
- [9] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Cham, Switzerland: Springer, 2022.
- [10] Roboflow Universe Projects, “License Plate Recognition Dataset (v13),” Roboflow Universe, CC BY 4.0, 2026. [En línea]. Disponible: <https://universe.roboflow.com/roboflow-universe-projects/license-plate-recognition-rxg4e/dataset/13>
- [11] Roboflow Universe, “Dataset OCR Placas,” Roboflow Universe, CC BY 4.0, 2026. [En línea].
- [12] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [13] E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *Int. J. Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.
- [14] T. Takagi and M. Sugeno, “Fuzzy identification of systems and its applications to modeling and control,” *IEEE Trans. Systems, Man, and Cybernetics*, vol. SMC-15, no. 1, pp. 116–132, 1985.
- [15] Presidencia de la República del Ecuador, “Reglamento a la Ley de Transporte Terrestre, Tránsito y Seguridad Vial,” Registro Oficial Suplemento 731, Ecuador, 2012, última reforma 2017.
- [16] J. Wang, C. Zhao, and Z. Liu, “Can historical accident data improve sustainable urban traffic safety? A predictive modeling study,” *Sustainability*, vol. 16, Art. no. 9642, 2024.